Curso Tecnologia em ADS
Sistemas Operacionais

Laboratório: Laboratório de Problemas Clássicos - Parte 1

Professor: Eraldo Silveira e Silva — Material elaborado com suporte do ChatGPT

1 Produtores e consumidores: solução clássica

Implemente o problema do produtor–consumidor com **múltiplos produtores e múltiplos consumidores**, utilizando um buffer circular compartilhado de capacidade limitada.

Requisitos

a) Crie inicialmente 3 threads produtoras e 3 consumidoras, com buffer de 5 posições.

b) Cada produtor deve gerar itens identificáveis (por exemplo, um par formado pelo identificador do produtor e um número sequencial).

c) Utilize dois **semáforos contadores**, um para vagas disponíveis e outro para itens disponíveis, além de um **mutex** para proteger a manipulação do buffer e seus índices.

d) Garanta que nenhum item seja consumido duas vezes ou sobrescrito antes do consumo.

e) Faça cada produtor gerar 20 itens. Distribua entre os consumidores a retirada de todos os itens produzidos, garantindo que as threads terminem sem ficarem bloqueadas indefinidamente.

f) Exiba, a cada inserção e retirada, a identificação da thread, o item e a ocupação do buffer. Garanta que os registros correspondam a estados consistentes.

Questões

a) Qual é a função de cada um dos três mecanismos de sincronização?

b) O que acontece quando o buffer fica cheio? E quando fica vazio?

c) Por que somente o mutex não é suficiente para implementar convenientemente a espera por vagas e itens?

d) O uso do mutex impede que produtores e consumidores realizem trabalho fora da região crítica simultaneamente? Justifique.

2 Leitores e escritores: solução simplista

Implemente um programa com 10 threads leitoras e 1 thread escritora que compartilham uma variável inteira. Cada leitor consulta o valor; o escritor o modifica periodicamente.

Requisitos

a) Utilize **um único mutex** para proteger toda operação de leitura ou escrita da região compartilhada.

b) Simule uma operação de leitura relativamente demorada, mantendo o mutex adquirido durante a leitura (por exemplo, com uma pequena pausa).

c) Apresente mensagens indicando quando cada thread entra e sai da região crítica.

d) Execute o programa com diferentes números de leitores e observe o comportamento.

Questões

a) A implementação garante a integridade da região compartilhada?

b) Dois leitores conseguem ler simultaneamente? Por quê?

c) Por que essa solução é considerada *simplista*, mesmo sendo correta quanto à exclusão mútua?

d) Que oportunidade de concorrência poderia ser explorada por uma solução mais elaborada?

3 Leitores e escritores: preferência aos leitores

Implemente o algoritmo clássico de preferência aos leitores, permitindo que vários leitores acessem simultaneamente a região compartilhada, mas exigindo exclusividade para o escritor. **Observação de implementação POSIX:** use um mutex para proteger o contador de leitores ativos e um **semáforo binário** para controlar o acesso à área compartilhada. O primeiro leitor adquire esse semáforo e o último o libera; como podem ser threads diferentes, não se deve substituir diretamente esse semáforo por um `pthread_mutex_t`.

Versão A — funcionamento normal

a) Crie 10 leitores e 1 escritor.

b) Mantenha um contador de leitores ativos, protegido por mutex.

c) O primeiro leitor deve impedir o acesso do escritor; o último deve liberá-lo.

d) O escritor deve obter acesso exclusivo antes de modificar o dado compartilhado.

e) Ajuste os intervalos de leitura e escrita para que seja possível observar tanto leituras simultâneas quanto escritas efetivamente realizadas.

f) Registre a entrada e a saída dos leitores, o número de leitores ativos e as operações do escritor.

Versão B — investigação de starvation do escritor

Partindo da versão A, modifique o padrão de chegada e a duração das leituras para manter leitores sobrepostos por longos períodos, mesmo enquanto o escritor aguarda.

a) Gere acessos sucessivos dos leitores, buscando manter o contador de leitores ativos maior que zero.

b) Registre o instante em que o escritor solicita acesso e o instante em que consegue entrar, utilizando, por exemplo, `clock_gettime(CLOCK_MONOTONIC, ...)`.

c) Compare o tempo de espera do escritor nas versões A e B.

d) Apresente um trecho do registro que mostre novos leitores entrando enquanto o escritor aguarda.

Questões

a) Por que o escritor pode permanecer bloqueado mesmo que leitores individuais terminem suas operações?

b) A chegada de novos leitores é impedida quando há um escritor aguardando?

c) Um tempo de espera muito longo demonstra, por si só, inanição indefinida? Explique a diferença entre evidência experimental e possibilidade teórica de *starvation*.

d) Proponha uma alteração no protocolo que impeça a entrada indefinida de novos leitores quando houver um escritor aguardando.

Entrega

Entregue os arquivos-fonte das três atividades (incluindo as duas versões do Exercício 3), instruções de compilação e um breve relatório com as respostas e registros de execução relevantes. Utilize, por exemplo, `gcc -pthread` para compilar os programas.